Programación II

Estructuras lineales

Tomás Capretto

FCEyE · Universidad Nacional de Rosario

Introducción

Introducción

En el apunte anterior analizamos el desempeño de distintas estructuras basadas en arreglos.

Introducción

En el apunte anterior analizamos el desempeño de distintas estructuras basadas en arreglos.

Pero la velocidad no es lo único que importa...

Introducción

En el apunte anterior analizamos el desempeño de distintas estructuras basadas en arreglos.

Pero la velocidad no es lo único que importa...

> Una estructura también puede ayudarnos a escribir código más simple y fácil de leer.

Estructuras lineales

Los elementos están organizados uno detrás de otro, formando una secuencia.

Cada elemento, excepto el primero y el último, tiene un predecesor y un sucesor.

Estructuras lineales

Los elementos están organizados uno detrás de otro, formando una secuencia.

Cada elemento, excepto el primero y el último, tiene un predecesor y un sucesor.

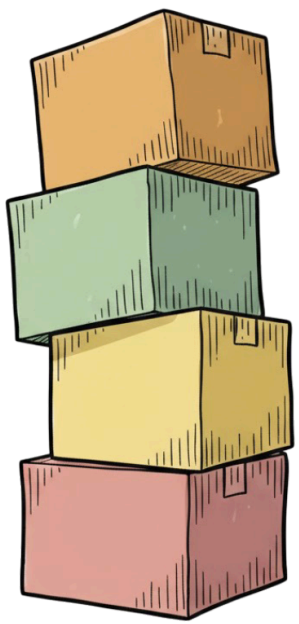
Vamos a trabajar con:

- Pilas
- Colas
- Listas enlazadas
- Listas doblemente enlazadas

Pilas

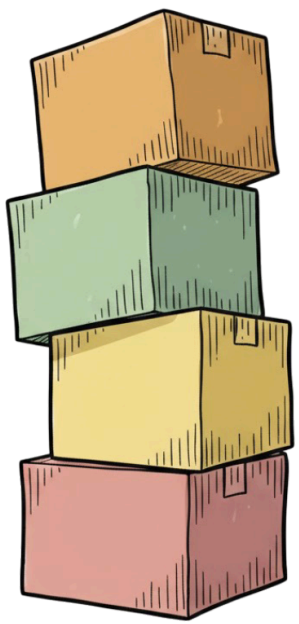
Pilas

Una colección de objetos con restricciones para insertar y eliminar.



Pilas

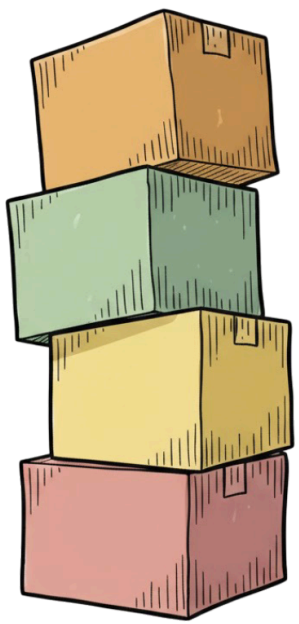
Una colección de objetos con restricciones para insertar y eliminar.



Primero colocamos la caja roja, luego la amarilla, la verde y la naranja.

Pilas

Una colección de objetos con restricciones para insertar y eliminar.

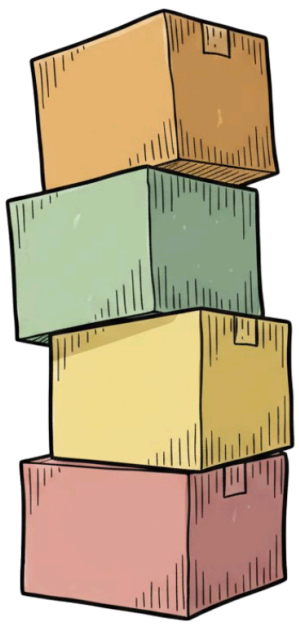


Primero colocamos la caja roja, luego la amarilla, la verde y la naranja.

¿Podemos sacar cualquiera?

Pilas

Una colección de objetos con restricciones para insertar y eliminar.

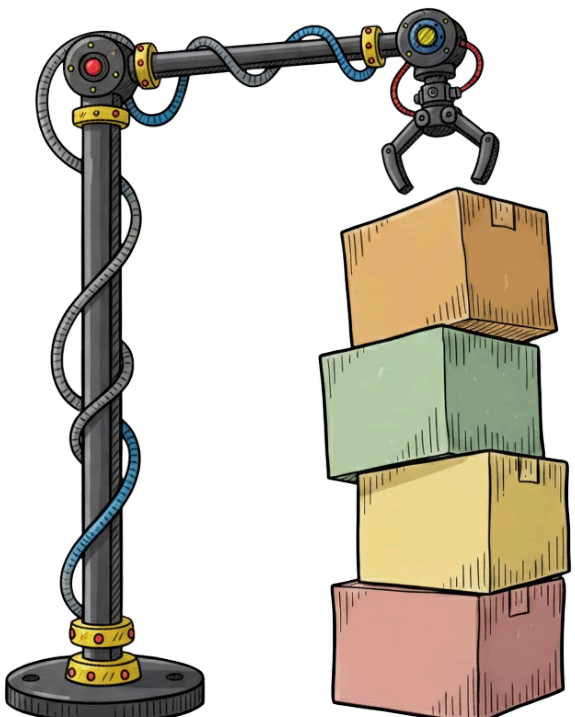


Primero colocamos la caja roja, luego la amarilla, la verde y la naranja.

¿Podemos sacar cualquiera?

Para sacar una caja, empezamos por la de arriba.

Pilas



Solo podemos leer, quitar o agregar cajas desde la cima.

Pilas

> **LIFO**
>
> *Last in, first out*: el último en entrar es el primero en salir.

Pilas

> **LIFO**
>
> *Last in, first out*: el último en entrar es el primero en salir.

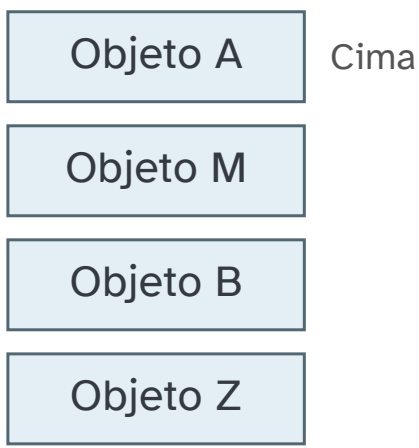
- Agregamos elementos en la cima.
- Leemos el elemento de la cima.
- Extraemos el elemento de la cima.

Pilas



Cada caja representa un objeto en memoria. Los colores nos ayudan a reconocerlos.

Pilas



Ninguna celda tiene propiedades especiales. Usamos el mismo color para todas. Solo podemos interactuar con la cima.

Lectura

Solo podemos leer el valor en la cima de la pila.

Lectura

Solo podemos leer el valor en la cima de la pila.

Leer no implica extraer el objeto.

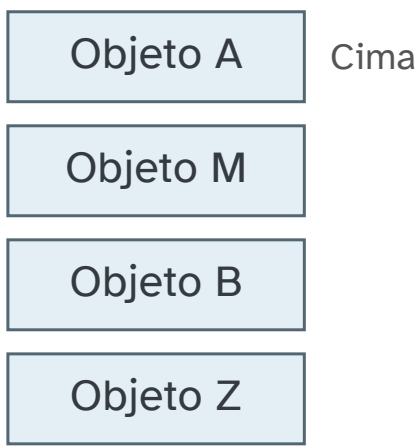
Lectura

Solo podemos leer el valor en la cima de la pila.

Leer no implica extraer el objeto.

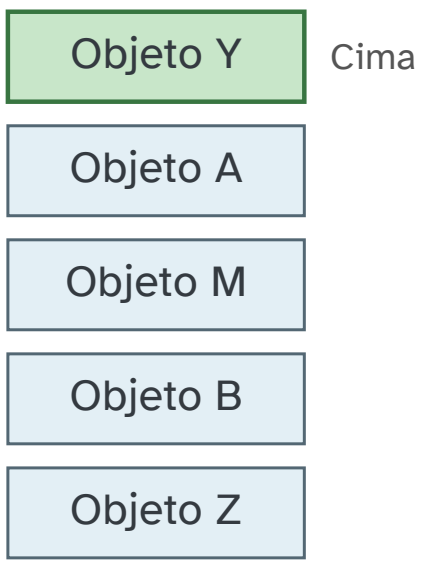
Para acceder a uno que está más abajo, primero debemos extraer los que tiene encima.

Inserción



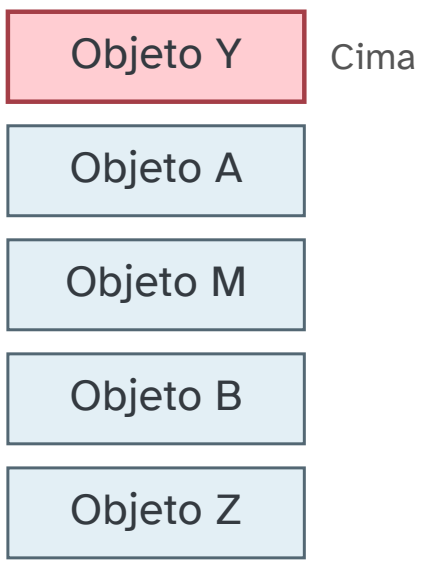
Queremos agregar un nuevo objeto a la pila.

Inserción



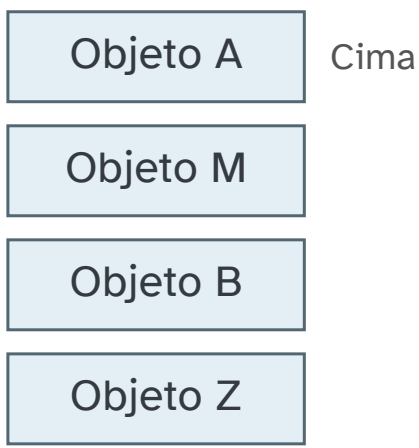
El nuevo objeto se coloca encima de todos los demás.

Eliminación



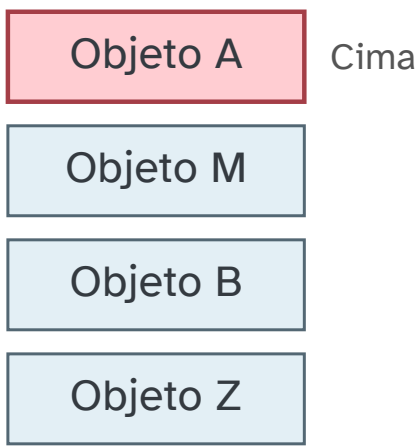
Solo podemos extraer el objeto en la cima: primero sale Objeto Y.

Eliminación



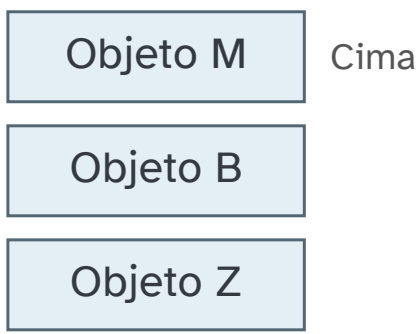
Después de extraer Objeto Y, Objeto A queda en la cima.

Eliminación



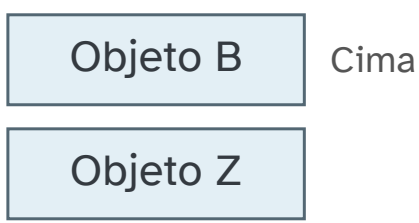
Para extraer varios objetos, repetimos el proceso desde la cima.

Eliminación



Extraemos Objeto A. Ahora Objeto M queda en la cima.

Eliminación



Al extraer Objeto M, Objeto B pasa a ocupar la cima.

Implementación

En la práctica, no trabajamos directamente con una pila de celdas de memoria.

Implementación

En la práctica, no trabajamos directamente con una pila de celdas de memoria.

La pila es un **tipo de dato abstracto**: define cómo podemos interactuar con los datos.

Implementación

En la práctica, no trabajamos directamente con una pila de celdas de memoria.

La pila es un **tipo de dato abstracto**: define cómo podemos interactuar con los datos.

Para usarla en un programa necesitamos una implementación concreta.

Implementación

Una opción: un arreglo al que le imponemos ciertas restricciones.

Implementación

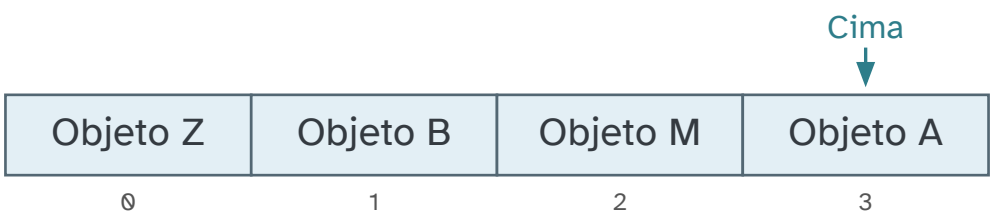
Una opción: un arreglo al que le imponemos ciertas restricciones.

La idea

Creamos una clase `Pila` que guarda los valores en una lista de Python.

Solo permitimos interactuar con el último elemento de esa lista.

Implementación



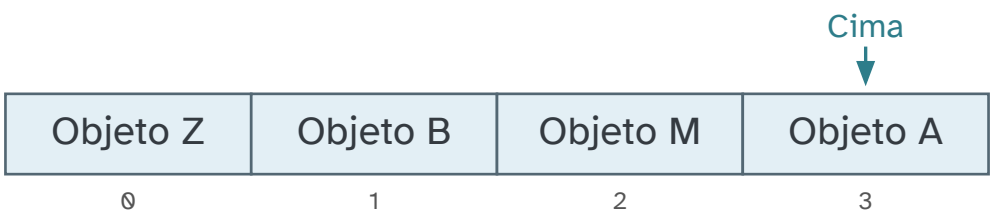
Imaginemos que tumbamos la pila: la cima corresponde al final del arreglo.

Implementación



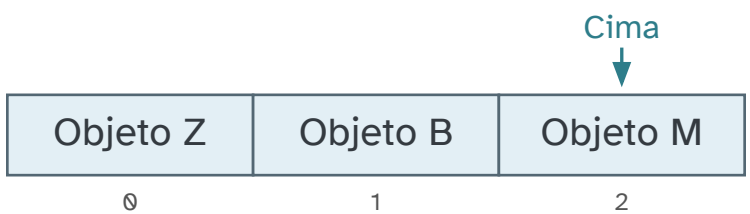
Insertar en la cima equivale a agregar al final del arreglo.

Implementación



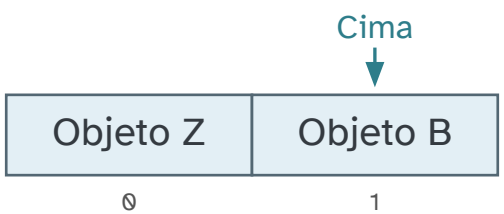
Extraer de la cima equivale a quitar el último elemento.

Implementación



La siguiente extracción vuelve a ocurrir al final.

Implementación



En cada extracción, la longitud del arreglo se reduce.

Implementación

```
class Pila:
    def __init__(self):
        self._datos = []
```

Implementación

```
class Pila:
    def __init__(self):
        self._datos = []

    def insertar(self, element):
        self._datos.append(element)
```

Implementación

```
class Pila:
    def __init__(self):
        self._datos = []

    def insertar(self, element):
        self._datos.append(element)

    def extraer(self):
        if len(self._datos) > 0:
            return self._datos.pop()
        return None
```

Implementación

```
class Pila:
    def __init__(self):
        self._datos = []

    def insertar(self, element):
        self._datos.append(element)

    def extraer(self):
        if len(self._datos) > 0:
            return self._datos.pop()
        return None
```

No indicamos una posición: siempre insertamos y extraemos desde la cima.

Implementación

Los otros métodos de `Pila`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[-1]
        return None
```

Implementación

Los otros métodos de `Pila`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[-1]
        return None

    @property
    def vacia(self):
        return len(self._datos) == 0
```

Implementación

Los otros métodos de `Pila`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[-1]
        return None

    @property
    def vacia(self):
        return len(self._datos) == 0
```

`leer` consulta la cima sin retirar el elemento.

Aplicaciones

El protocolo LIFO nos permite resolver distintos problemas.

- Invertir el orden de los datos.

Aplicaciones

El protocolo LIFO nos permite resolver distintos problemas.

- Invertir el orden de los datos.
- Verificar paréntesis y corchetes.

Invertir orden

Apilamos los valores 1, 2 y 3, en ese orden.

Invertir orden

Apilamos los valores 1, 2 y 3, en ese orden.

Al extraerlos, obtenemos 3, 2 y 1.

Invertir orden

Apilamos los valores 1, 2 y 3, en ese orden.

Al extraerlos, obtenemos 3, 2 y 1.

> Podemos hacer lo mismo con las líneas de un archivo: apilarlas y escribirlas en el orden en que salen.

Invertir orden

```
def invertir_archivo(origen, destino):
    pila = Pila()
```

Invertir orden

```
def invertir_archivo(origen, destino):
    pila = Pila()

    with open(origen) as archivo_origen:
        for linea in archivo_origen:
            pila.insertar(linea.rstrip("\n"))
```

Invertir orden

```python
def invertir_archivo(origen, destino):
    pila = Pila()

    with open(origen) as archivo_origen:
        for linea in archivo_origen:
            pila.insertar(linea.rstrip("\n"))

    with open(destino, "w") as archivo_destino:
        while not pila.vacia:
            archivo_destino.write(pila.extraer() + "\n")
```

Invertir orden

```python
def invertir_archivo(origen, destino):
    pila = Pila()

    with open(origen) as archivo_origen:
        for linea in archivo_origen:
            pila.insertar(linea.rstrip("\n"))

    with open(destino, "w") as archivo_destino:
        while not pila.vacia:
            archivo_destino.write(pila.extraer() + "\n")
```

Primero apilamos todas las líneas. Después las extraemos una a una.

Invertir orden

`original.txt`:

```
¿Quién es tu nuevo amor?
¿Tu nueva ocupación?
¿Tu misterioso alguien?
```

Invertir orden

`original.txt`:

```
¿Quién es tu nuevo amor?
¿Tu nueva ocupación?
¿Tu misterioso alguien?
```

```
invertir_archivo("original.txt", "invertido.txt")
```

Invertir orden

`original.txt`:

```
¿Quién es tu nuevo amor?
¿Tu nueva ocupación?
¿Tu misterioso alguien?
```

```
invertir_archivo("original.txt", "invertido.txt")
```

`invertido.txt`:

```
¿Tu misterioso alguien?
¿Tu nueva ocupación?
¿Quién es tu nuevo amor?
```

Verificar paréntesis y corchetes

Cada símbolo de apertura debe tener su correspondiente cierre.

Verificar paréntesis y corchetes

Cada símbolo de apertura debe tener su correspondiente cierre.

¿Alcanza con contar cuántos abren y cuántos cierran?

```
1 +) (3 * 5()
```

Verificar paréntesis y corchetes

Cada símbolo de apertura debe tener su correspondiente cierre.

¿Alcanza con contar cuántos abren y cuántos cierran?

```
1 +) (3 * 5()
```

Hay la misma cantidad, pero la expresión es incorrecta.

Verificar paréntesis y corchetes

Recorremos la expresión de izquierda a derecha.

- Si encontramos una apertura, la apilamos.

Verificar paréntesis y corchetes

Recorremos la expresión de izquierda a derecha.

- Si encontramos una apertura, la apilamos.
- Si encontramos un cierre, extraemos la última apertura.

Verificar paréntesis y corchetes

Recorremos la expresión de izquierda a derecha.

- Si encontramos una apertura, la apilamos.
- Si encontramos un cierre, extraemos la última apertura.
- Verificamos que formen un par válido.

Verificar paréntesis y corchetes

Recorremos la expresión de izquierda a derecha.

- Si encontramos una apertura, la apilamos.
- Si encontramos un cierre, extraemos la última apertura.
- Verificamos que formen un par válido.
- Al terminar, la pila debe estar vacía.

Verificar paréntesis y corchetes

```
def verificar_agrupamientos(expr):
    apertura = "(["
    cierre = ")]"
    pila = Pila()
```

Verificar paréntesis y corchetes

```
def verificar_agrupamientos(expr):
    apertura = "(["
    cierre = ")]"
    pila = Pila()

    for caracter in expr:
        if caracter in apertura:
            pila.insertar(caracter)
```

Verificar paréntesis y corchetes

```
def verificar_agrupamientos(expr):
    apertura = "(["
    cierre = ")]"
    pila = Pila()

    for caracter in expr:
        if caracter in apertura:
            pila.insertar(caracter)
        elif caracter in cierre:
            if pila.vacia:
                return False
```

Verificar paréntesis y corchetes

```
def verificar_agrupamientos(expr):
    apertura = "(["
    cierre = ")]"
    pila = Pila()

    for caracter in expr:
        if caracter in apertura:
            pila.insertar(caracter)
        elif caracter in cierre:
            if pila.vacia:
                return False
            if cierre.index(caracter) ≠ apertura.index(pila.extraer()):
                return False
```

Verificar paréntesis y corchetes

```
def verificar_agrupamientos(expr):
    apertura = "(["
    cierre = ")]"
    pila = Pila()

    for caracter in expr:
        if caracter in apertura:
            pila.insertar(caracter)
        elif caracter in cierre:
            if pila.vacia:
                return False
            if cierre.index(caracter) ≠ apertura.index(pila.extraer()):
                return False

    return pila.vacia
```

Verificar paréntesis y corchetes

Por ejemplo, con `[3 + (4 * 5)]`:

- Leemos `[` ⟶ lo apilamos.

Verificar paréntesis y corchetes

Por ejemplo, con `[3 + (4 * 5)]`:

- Leemos `[` ⟶ lo apilamos.
- Leemos `(` ⟶ lo apilamos encima del corchete.

Verificar paréntesis y corchetes

Por ejemplo, con `[3 + (4 * 5)]`:

- Leemos `[` ⟶ lo apilamos.
- Leemos `(` ⟶ lo apilamos encima del corchete.
- Leemos `)` ⟶ extraemos `(`. Forman un par.

Verificar paréntesis y corchetes

Por ejemplo, con `[3 + (4 * 5)]`:

- Leemos `[` ⟶ lo apilamos.
- Leemos `(` ⟶ lo apilamos encima del corchete.
- Leemos `)` ⟶ extraemos `(`. Forman un par.
- Leemos `]` ⟶ extraemos `[`. Forman un par.

Verificar paréntesis y corchetes

Por ejemplo, con `[3 + (4 * 5)]`:

- Leemos `[` ⟶ lo apilamos.
- Leemos `(` ⟶ lo apilamos encima del corchete.
- Leemos `)` ⟶ extraemos `(`. Forman un par.
- Leemos `]` ⟶ extraemos `[`. Forman un par.
- Terminamos con la pila vacía ⟶ `True`.

Verificar paréntesis y corchetes

```
verificar_agrupamientos("(3 + (4 * 5))")  # True
verificar_agrupamientos("[3 + (4 * 5)]")  # True
```

Verificar paréntesis y corchetes

```
verificar_agrupamientos("(3 + (4 * 5))")  # True
verificar_agrupamientos("[3 + (4 * 5)]")  # True
```

```
verificar_agrupamientos("(3 + (4 * 5)")   # False
verificar_agrupamientos("(3 + [(4 * 5)")  # False
```

Verificar paréntesis y corchetes

```
verificar_agrupamientos("(3 + (4 * 5))")  # True
verificar_agrupamientos("[3 + (4 * 5)]")  # True
```

```
verificar_agrupamientos("(3 + (4 * 5)")   # False
verificar_agrupamientos("(3 + [(4 * 5)")  # False
```

Si queda una apertura sin cerrar, la pila no queda vacía.

Colas

Colas



Los primeros en llegar al banco son los primeros en entrar.

Colas

Una colección ordenada de objetos que sigue el principio FIFO.

> **FIFO**
>
> *First in, first out*: el primero en entrar es el primero en salir.

Colas

Una colección ordenada de objetos que sigue el principio FIFO.

> **FIFO**
>
> *First in, first out*: el primero en entrar es el primero en salir.

- Los nuevos elementos se agregan al final.
- Los elementos se extraen desde el frente.

Colas

Las solicitudes se atienden en orden de llegada.

Algunos ejemplos:

- Restaurantes y tiendas.

Colas

Las solicitudes se atienden en orden de llegada.

Algunos ejemplos:

- Restaurantes y tiendas.
- Plataformas de venta de entradas.

Colas

Las solicitudes se atienden en orden de llegada.

Algunos ejemplos:

- Restaurantes y tiendas.
- Plataformas de venta de entradas.
- Impresoras en red.

Colas

Las solicitudes se atienden en orden de llegada.

Algunos ejemplos:

- Restaurantes y tiendas.
- Plataformas de venta de entradas.
- Impresoras en red.
- Atención de llamadas.

Implementación

La cola también es un **tipo de dato abstracto**.

Define cómo se comporta, pero no cómo almacena los elementos.

Implementación

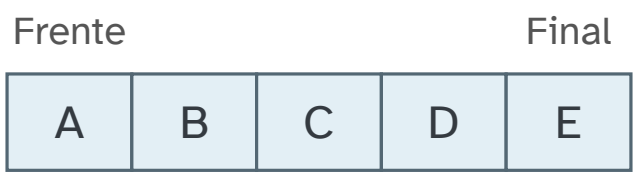
La cola también es un **tipo de dato abstracto**.

Define cómo se comporta, pero no cómo almacena los elementos.

Una implementación posible usa un arreglo restringido:

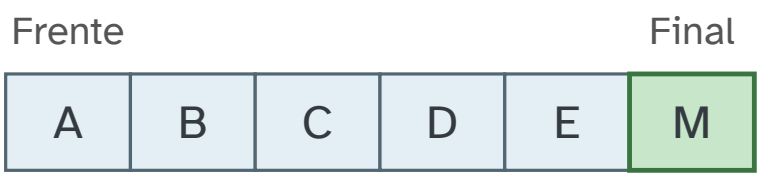
- Insertamos al final.
- Extraemos desde el índice `0`.

Colas



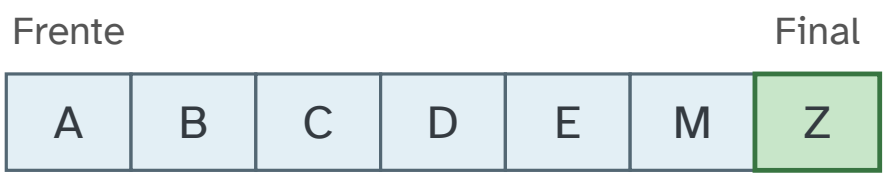
Los elementos entran por el final y salen por el frente.

Colas



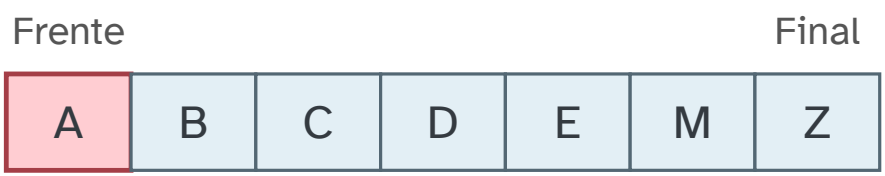
Agregamos M al final de la cola.

Colas



Si agregamos Z, también lo hacemos al final.

Colas



Extraemos A, el primero en haber llegado.

Colas



B pasa a ocupar el frente de la cola.

Colas



En la siguiente extracción sale B.

Colas



Los demás avanzan hacia el frente. El orden de llegada se conserva.

Implementación

```
class Cola:
    def __init__(self):
        self._datos = []
```

Implementación

```
class Cola:
    def __init__(self):
        self._datos = []

    def insertar(self, element):  # enqueue
        self._datos.append(element)
```

Implementación

```
class Cola:
    def __init__(self):
        self._datos = []

    def insertar(self, element):  # enqueue
        self._datos.append(element)

    def extraer(self):            # dequeue
        if len(self._datos) > 0:
            return self._datos.pop(0)
        return None
```

Implementación

```
class Cola:
    def __init__(self):
        self._datos = []

    def insertar(self, element):  # enqueue
        self._datos.append(element)

    def extraer(self):            # dequeue
        if len(self._datos) > 0:
            return self._datos.pop(0)
        return None
```

A diferencia de la pila, extraemos desde el frente.

Implementación

Los otros métodos de `Cola`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[0]
        return None
```

Implementación

Los otros métodos de `Cola`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[0]
        return None

    @property
    def vacia(self):
        return len(self._datos) == 0
```

Implementación

Los otros métodos de `Cola`:

```
    def leer(self):
        if len(self._datos) > 0:
            return self._datos[0]
        return None

    @property
    def vacia(self):
        return len(self._datos) == 0

    def __len__(self):
        return len(self._datos)
```

Cola de impresión

```
cola_impresion = Cola()
cola_impresion.insertar("documento1.pdf")
cola_impresion.insertar("documento2.docx")
cola_impresion.insertar("documento3.xlsx")
```

Cola de impresión

```
cola_impresion = Cola()
cola_impresion.insertar("documento1.pdf")
cola_impresion.insertar("documento2.docx")
cola_impresion.insertar("documento3.xlsx")

while not cola_impresion.vacia:
    print(f"Imprimiendo {cola_impresion.extraer()} ... ")
```

Cola de impresión

```
cola_impresion = Cola()
cola_impresion.insertar("documento1.pdf")
cola_impresion.insertar("documento2.docx")
cola_impresion.insertar("documento3.xlsx")

while not cola_impresion.vacia:
    print(f"Imprimiendo {cola_impresion.extraer()} ... ")

print(cola_impresion.vacia)
```

Cola de impresión

```
cola_impresion = Cola()
cola_impresion.insertar("documento1.pdf")
cola_impresion.insertar("documento2.docx")
cola_impresion.insertar("documento3.xlsx")

while not cola_impresion.vacia:
    print(f"Imprimiendo {cola_impresion.extraer()} ... ")

print(cola_impresion.vacia)
```

```
Imprimiendo documento1.pdf ...
Imprimiendo documento2.docx ...
Imprimiendo documento3.xlsx ...
True
```

Atención de llamadas

Simulamos un centro de llamadas con clientes en espera.

- Si llega una llamada, la agregamos al final de la cola.

Atención de llamadas

Simulamos un centro de llamadas con clientes en espera.

- Si llega una llamada, la agregamos al final de la cola.
- Si hay alguien disponible, atendemos la primera llamada.

Atención de llamadas

Simulamos un centro de llamadas con clientes en espera.

- Si llega una llamada, la agregamos al final de la cola.
- Si hay alguien disponible, atendemos la primera llamada.
- Si no ocurre ninguna de las dos cosas, esperamos.

Atención de llamadas

Simulamos un centro de llamadas con clientes en espera.

- Si llega una llamada, la agregamos al final de la cola.
- Si hay alguien disponible, atendemos la primera llamada.
- Si no ocurre ninguna de las dos cosas, esperamos.

Ver la simulación del apunte ↗

Atención de llamadas

El núcleo de la simulación:

```
accion = random.random()
ahora = datetime.now().strftime("%H:%M:%S")
```

Atención de llamadas

El núcleo de la simulación:

```
accion = random.random()
ahora = datetime.now().strftime("%H:%M:%S")

if accion < 0.5:
    llamada = {"cliente": random.choice(nombres), "hora": ahora}
    cola.insertar(llamada)
    print(f"[{ahora}] Nueva llamada de {llamada['cliente']}")
```

Atención de llamadas

El núcleo de la simulación:

```
accion = random.random()
ahora = datetime.now().strftime("%H:%M:%S")

if accion < 0.5:
    llamada = {"cliente": random.choice(nombres), "hora": ahora}
    cola.insertar(llamada)
    print(f"[{ahora}] Nueva llamada de {llamada['cliente']}")
elif accion < 0.9 and not cola.vacia:
    llamada = cola.extraer()
    print(f"[{ahora}] Atendiendo a {llamada['cliente']}")
```

Atención de llamadas

El núcleo de la simulación:

```
accion = random.random()
ahora = datetime.now().strftime("%H:%M:%S")

if accion < 0.5:
    llamada = {"cliente": random.choice(nombres), "hora": ahora}
    cola.insertar(llamada)
    print(f"[{ahora}] Nueva llamada de {llamada['cliente']}")
elif accion < 0.9 and not cola.vacia:
    llamada = cola.extraer()
    print(f"[{ahora}] Atendiendo a {llamada['cliente']}")
else:
    print(f"[{ahora}] Sin actividad ... ")
```

Atención de llamadas

El núcleo de la simulación:

```
accion = random.random()
ahora = datetime.now().strftime("%H:%M:%S")

if accion < 0.5:
    llamada = {"cliente": random.choice(nombres), "hora": ahora}
    cola.insertar(llamada)
    print(f"[{ahora}] Nueva llamada de {llamada['cliente']}")
elif accion < 0.9 and not cola.vacia:
    llamada = cola.extraer()
    print(f"[{ahora}] Atendiendo a {llamada['cliente']}")
else:
    print(f"[{ahora}] Sin actividad ... ")
```

La cola conserva el orden de llegada de las llamadas.

Por qué usar estructuras de datos restringidas

Si para implementar una pila restringimos un arreglo...

Por qué usar estructuras de datos restringidas

Si para implementar una pila restringimos un arreglo...

> ¿Por qué no usar directamente el arreglo?

Por qué usar estructuras de datos restringidas

- **Evitar errores**: solo permitimos las operaciones que corresponden.

Por qué usar estructuras de datos restringidas

- **Evitar errores**: solo permitimos las operaciones que corresponden.
- **Un modelo mental claro**: LIFO o FIFO orientan la solución.

Por qué usar estructuras de datos restringidas

- **Evitar errores**: solo permitimos las operaciones que corresponden.
- **Un modelo mental claro**: LIFO o FIFO orientan la solución.
- **Código más legible**: quien ve una pila sabe en qué orden salen los datos.

Listas enlazadas

Listas enlazadas

Otra forma de mantener los elementos en un cierto orden.

Listas enlazadas

Otra forma de mantener los elementos en un cierto orden.

- Un arreglo guarda los datos en una región contigua de memoria.
- Una lista enlazada puede distribuirlos en distintas ubicaciones.

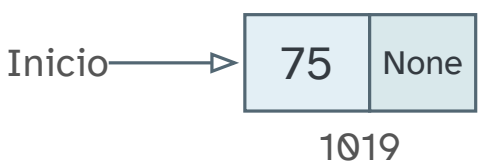
Listas enlazadas

Otra forma de mantener los elementos en un cierto orden.

- Un arreglo guarda los datos en una región contigua de memoria.
- Una lista enlazada puede distribuirlos en distintas ubicaciones.

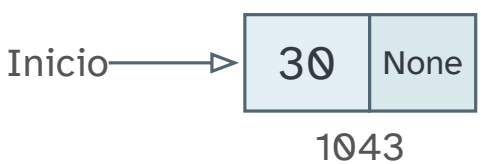
¿Cómo reconstruimos la secuencia si los elementos están separados?

El nodo



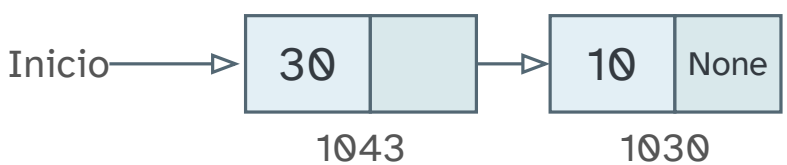
Cada nodo guarda un valor y una referencia al siguiente. Debajo mostramos su dirección en memoria. `None` indica que no hay un siguiente nodo.

El nodo



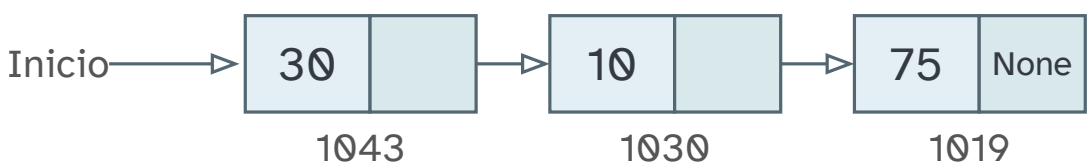
Creamos un primer nodo. Por ahora no tiene un siguiente.

El nodo



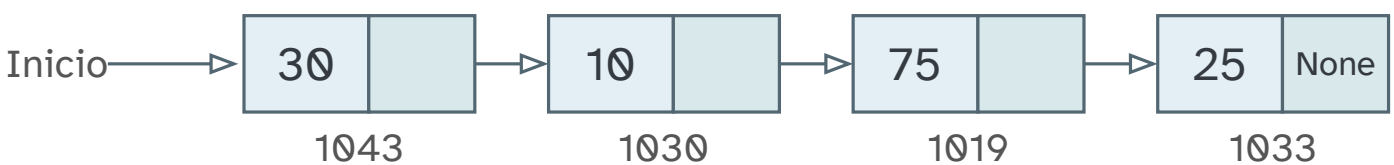
Enlazamos el primer nodo con el segundo.

El nodo



El segundo nodo apunta al tercero.

El nodo



El tercer nodo apunta al cuarto. El último no tiene siguiente.

El nodo

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.siguiente = None
```

El nodo

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.siguiente = None

    def __repr__(self):
        return f"Nodo({self.valor})"
```

El nodo

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.siguiente = None

    def __repr__(self):
        return f"Nodo({self.valor})"

Nodo(75)
# Nodo(75)
```

El nodo

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.siguiente = None

    def __repr__(self):
        return f"Nodo({self.valor})"

Nodo(75)
# Nodo(75)
```

Los enlaces entre nodos dan nombre a la **lista enlazada**.

El nodo

Creamos cuatro nodos independientes:

```
nodo1 = Nodo(30)
nodo2 = Nodo(10)
nodo3 = Nodo(75)
nodo4 = Nodo(25)
```

El nodo

Creamos cuatro nodos independientes:

```
nodo1 = Nodo(30)
nodo2 = Nodo(10)
nodo3 = Nodo(75)
nodo4 = Nodo(25)
```

Los enlazamos:

```
nodo1.siguiente = nodo2
nodo2.siguiente = nodo3
nodo3.siguiente = nodo4
```

El último nodo conserva `siguiente = None`.

El nodo

Para recorrerlos, seguimos los enlaces:

```
actual = nodo1
```

El nodo

Para recorrerlos, seguimos los enlaces:

```
actual = nodo1
while actual:
    print(actual, end=" → ")
    actual = actual.siguiente
```

El nodo

Para recorrerlos, seguimos los enlaces:

```
actual = nodo1
while actual:
    print(actual, end=" → ")
    actual = actual.siguiente
print("None")
```

El nodo

Para recorrerlos, seguimos los enlaces:

```
actual = nodo1
while actual:
    print(actual, end=" → ")
    actual = actual.siguiente
print("None")
```

```
Nodo(30) → Nodo(10) → Nodo(75) → Nodo(25) → None
```

Implementación básica

Solo necesitamos guardar una referencia al primer nodo.

```
class ListaEnlazada:
    def __init__(self, primer_nodo=None):
        self.primer_nodo = primer_nodo
```

Implementación básica

Solo necesitamos guardar una referencia al primer nodo.

```
class ListaEnlazada:
    def __init__(self, primer_nodo=None):
        self.primer_nodo = primer_nodo

lista = ListaEnlazada(nodo1)
```

Implementación básica

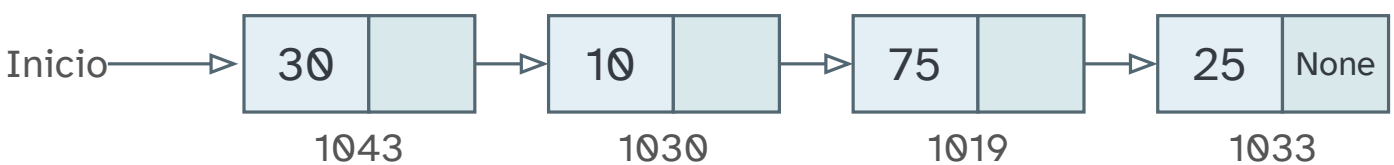
Solo necesitamos guardar una referencia al primer nodo.

```
class ListaEnlazada:
    def __init__(self, primer_nodo=None):
        self.primer_nodo = primer_nodo

lista = ListaEnlazada(nodo1)
```

Desde allí, podemos reconstruir toda la secuencia siguiendo los enlaces.

Implementación básica



Solo necesitamos guardar una referencia al primer nodo. Desde allí, seguimos los enlaces. Las direcciones no tienen que ser consecutivas.

Implementación básica

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 .
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1019	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

Los nodos pueden estar dispersos en la memoria. Cada celda representa espacio para un nodo; las direcciones son ilustrativas.

Implementación básica

1018	1019 **75** siguiente: 1033	1020	1021	1022	1023	1024
1025	1026	1027	1028	1029	1030 **10** siguiente: 1019	1031
1032	1033 **25** siguiente: None	1034	1035	1036	1037	1038
1039	1040	1041	1042	1043 **30** siguiente: 1030	1044	1045

La lista comienza en 1043. Ese nodo contiene el 30 y una referencia a 1030.

Implementación básica

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 .
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1019	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

Seguimos la referencia hasta 1030. El nodo con valor 10 apunta a 1019: los enlaces mantienen el orden de la secuencia.

Implementación básica

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 .
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1019	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

En 1019 encontramos el valor 75 y la referencia a 1033. Seguimos ese enlace.

Implementación básica

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 .
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1019	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

El nodo en 1033 contiene el 25. Su referencia siguiente es None: llegamos al final de la lista.

Lectura

Leer una dirección conocida de memoria sigue siendo $O(1)$.

Lectura

Leer una dirección conocida de memoria sigue siendo $O(1)$.

Pero ahora no podemos calcular la dirección de un elemento usando su índice.

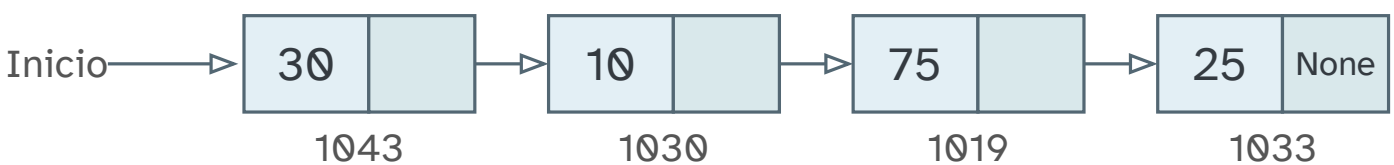
Lectura

Leer una dirección conocida de memoria sigue siendo $O(1)$.

Pero ahora no podemos calcular la dirección de un elemento usando su índice.

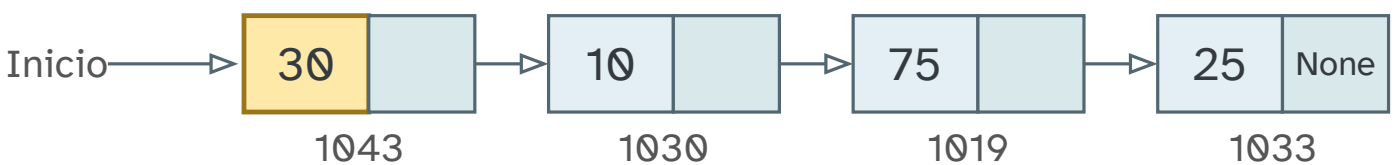
> Conocemos la dirección del primer nodo. Para llegar a los demás, tenemos que seguir los enlaces.

Lectura



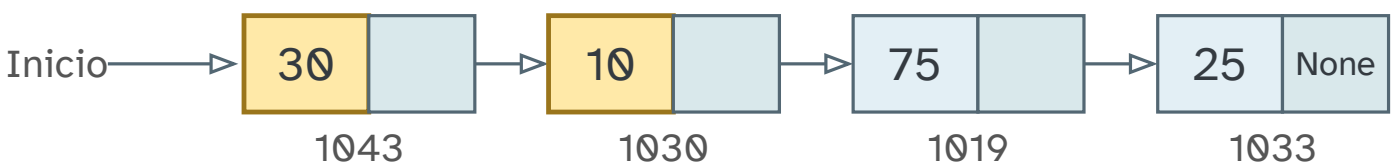
Queremos leer el valor del índice 3. Conocemos la dirección del primer nodo.

Lectura



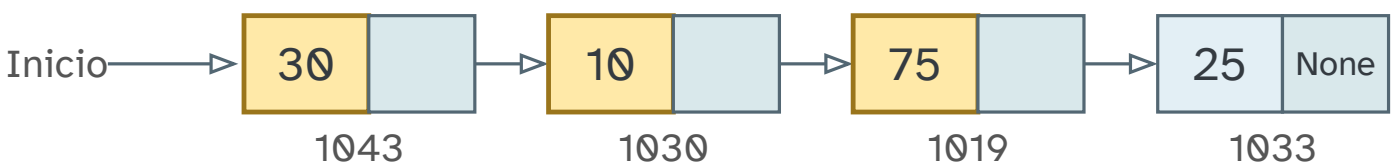
Empezamos en el primer nodo, índice 0.

Lectura



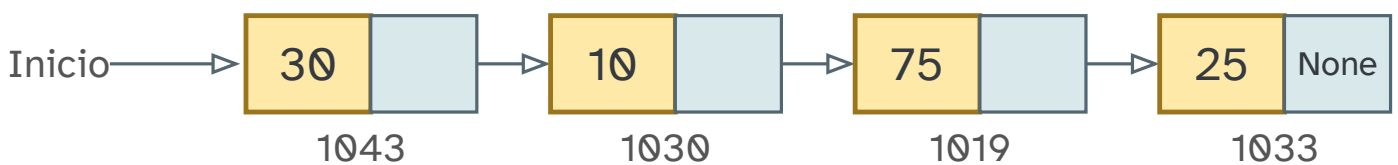
Seguimos el enlace al nodo del índice 1.

Lectura



Seguimos al índice 2. Todavía no llegamos al índice solicitado.

Lectura



Llegamos al índice 3 y devolvemos su valor: 25. Para llegar hasta allí pasamos por los nodos anteriores.

Lectura

Con N elementos, leer el último requiere recorrer toda la lista.

- Lectura en un arreglo: $O(1)$.
- Lectura por índice en una lista enlazada: $O(N)$.

Lectura

Con N elementos, leer el último requiere recorrer toda la lista.

- Lectura en un arreglo: $O(1)$.
- Lectura por índice en una lista enlazada: $O(N)$.

No son todas pálidas... Pronto vamos a ver sus puntos fuertes.

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
```

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
        nodo_actual = self.primer_nodo
        indice_actual = 0
```

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < indice:
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1
```

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < indice:
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None:
            return None
```

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < indice:
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None:
            return None
        return nodo_actual.valor
```

Lectura

```
    def leer(self, indice):
        if indice < 0:
            return None
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < indice:
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None:
            return None
        return nodo_actual.valor
```

Si no existe ese índice, devolvemos `None`.

Búsqueda

En lugar de recibir un índice y devolver un valor, recibimos un valor y devolvemos su índice.

- Comparamos el valor buscado con el del primer nodo.

Búsqueda

En lugar de recibir un índice y devolver un valor, recibimos un valor y devolvemos su índice.

- Comparamos el valor buscado con el del primer nodo.
- Si coincide, devolvemos el índice.

Búsqueda

En lugar de recibir un índice y devolver un valor, recibimos un valor y devolvemos su índice.

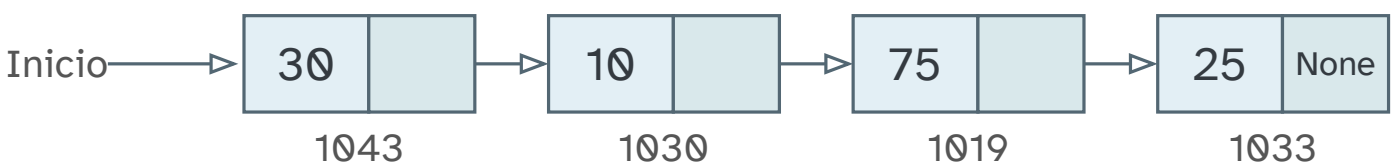
- Comparamos el valor buscado con el del primer nodo.
- Si coincide, devolvemos el índice.
- Si no coincide, avanzamos al siguiente nodo.

Búsqueda

En lugar de recibir un índice y devolver un valor, recibimos un valor y devolvemos su índice.

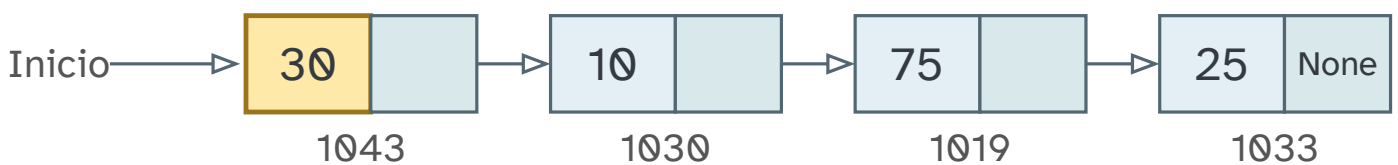
- Comparamos el valor buscado con el del primer nodo.
- Si coincide, devolvemos el índice.
- Si no coincide, avanzamos al siguiente nodo.
- Si llegamos al final, el valor no está en la lista.

Búsqueda



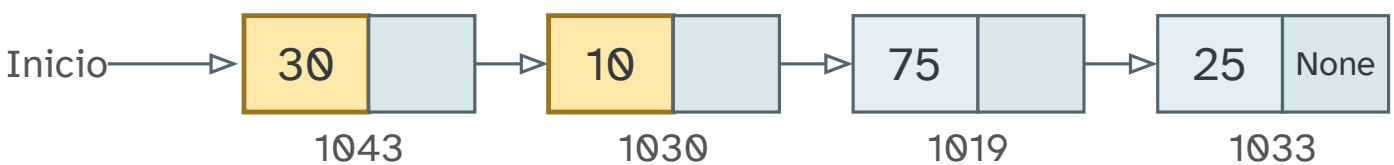
Recibimos un valor y queremos devolver su índice. Busquemos el 75.

Búsqueda



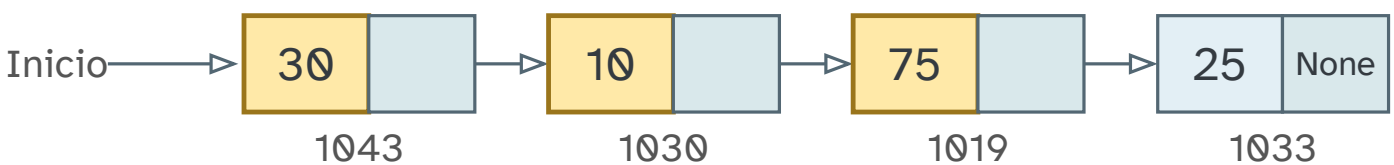
Comparamos el valor del primer nodo con el buscado. El 30 no coincide.

Búsqueda



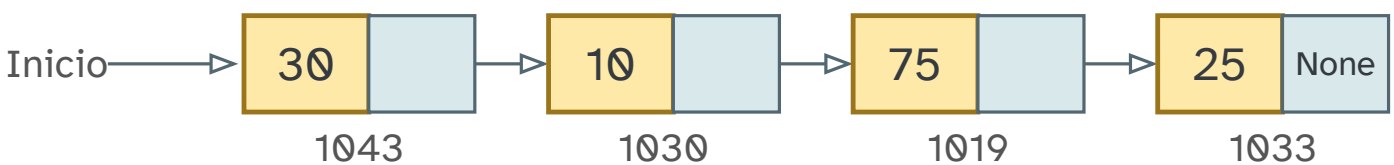
Avanzamos al siguiente nodo. El 10 tampoco coincide.

Búsqueda



Encontramos el 75 en el índice 2. La búsqueda termina.

Búsqueda



Si buscamos el 99, llegamos al final sin encontrarlo. Devolvemos None.

Búsqueda

```
    def buscar(self, valor):
        nodo_actual = self.primer_nodo
        indice_actual = 0
```

Búsqueda

```
    def buscar(self, valor):
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None:
            if nodo_actual.valor == valor:
                return indice_actual
```

Búsqueda

```
    def buscar(self, valor):
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None:
            if nodo_actual.valor == valor:
                return indice_actual
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1
```

Búsqueda

```
    def buscar(self, valor):
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None:
            if nodo_actual.valor == valor:
                return indice_actual
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        return None
```

Búsqueda

```
    def buscar(self, valor):
        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None:
            if nodo_actual.valor == valor:
                return indice_actual
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        return None
```

En el peor caso recorremos N nodos: $O(N)$, igual que la búsqueda lineal en un arreglo.

Inserción

Hasta ahora, el desempeño es igual o peor que el de un arreglo.

Inserción

Hasta ahora, el desempeño es igual o peor que el de un arreglo.

¿Qué pasa si queremos insertar al inicio?

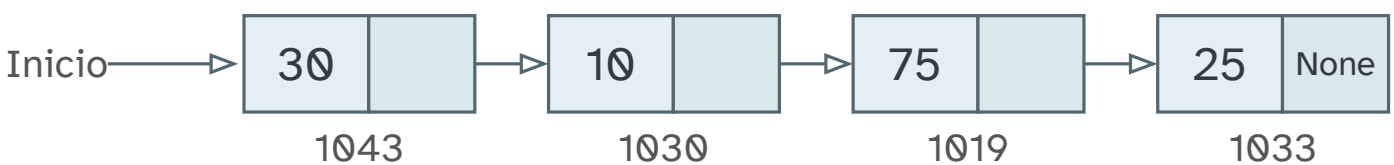
Inserción

Hasta ahora, el desempeño es igual o peor que el de un arreglo.

¿Qué pasa si queremos insertar al inicio?

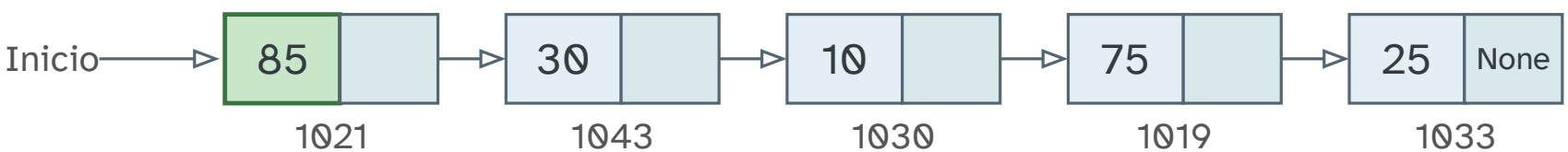
- En un arreglo, desplazamos todos los elementos.
- En una lista enlazada, creamos un nodo y actualizamos enlaces.

Inserción



Queremos insertar un nuevo elemento al inicio.

Inserción



El nuevo nodo apunta al primero anterior. La lista pasa a comenzar en el nuevo nodo. Insertar al inicio es de orden O(1).

Inserción

Insertar al inicio requiere una cantidad constante de operaciones.

```
nodo_nuevo = Nodo(valor)
nodo_nuevo.siguiente = self.primer_nodo
self.primer_nodo = nodo_nuevo
```

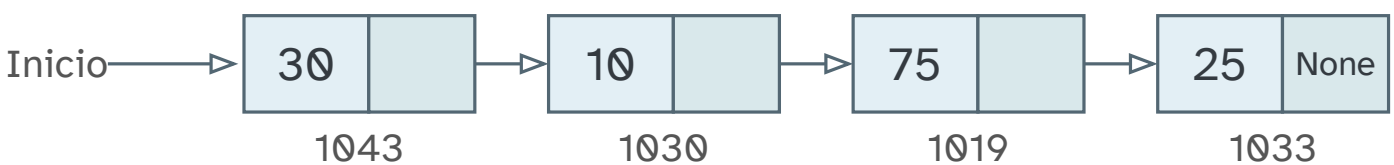
Inserción

Insertar al inicio requiere una cantidad constante de operaciones.

```
nodo_nuevo = Nodo(valor)
nodo_nuevo.siguiente = self.primer_nodo
self.primer_nodo = nodo_nuevo
```

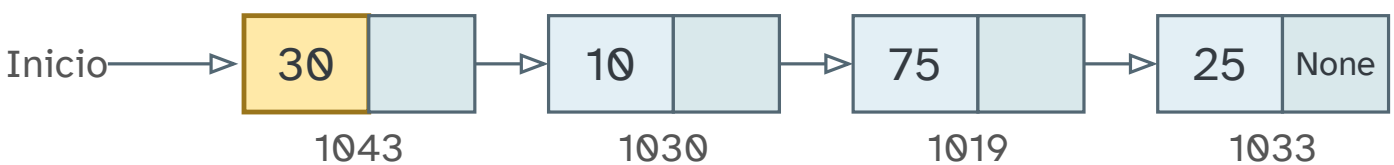
> No desplazamos los demás elementos: insertar al inicio es $O(1)$.

Inserción



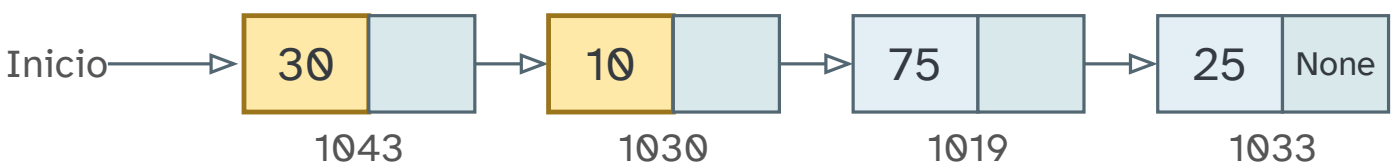
Queremos insertar 50 entre 10 y 75. Primero buscamos el nodo anterior.

Inserción



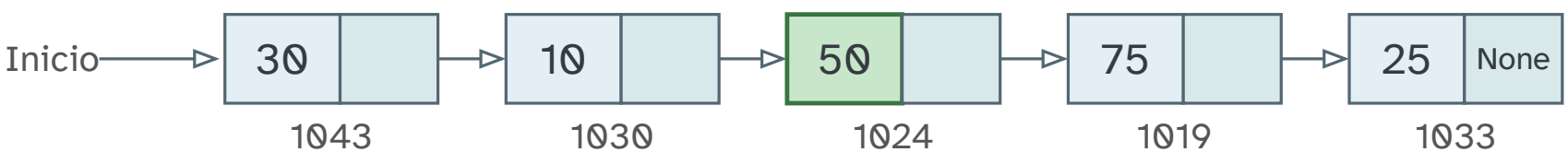
Comenzamos el recorrido desde el primer nodo.

Inserción



Encontramos el nodo anterior a la posición de inserción: el que contiene el 10.

Inserción



Actualizamos los enlaces: el nuevo nodo apunta al 75 y el nodo con valor 10 apunta al nuevo nodo.

Inserción

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 .
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1019	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

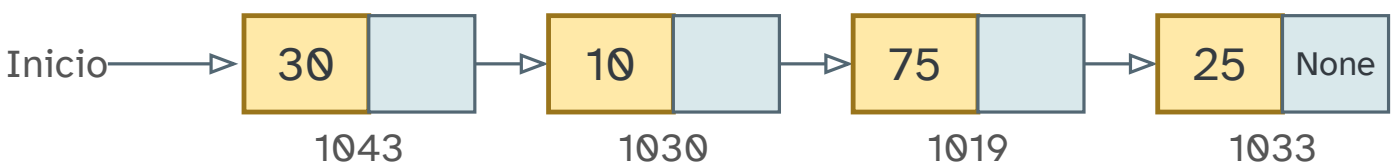
Antes de insertar, el nodo con valor 10 apunta al nodo con valor 75, en 1019.

Inserción

1018 .	1019 **75** siguiente: 1033	1020 .	1021 .	1022 .	1023 .	1024 **50** siguiente: 1019
1025 .	1026 .	1027 .	1028 .	1029 .	1030 **10** siguiente: 1024	1031 .
1032 .	1033 **25** siguiente: None	1034 .	1035 .	1036 .	1037 .	1038 .
1039 .	1040 .	1041 .	1042 .	1043 **30** siguiente: 1030	1044 .	1045 .

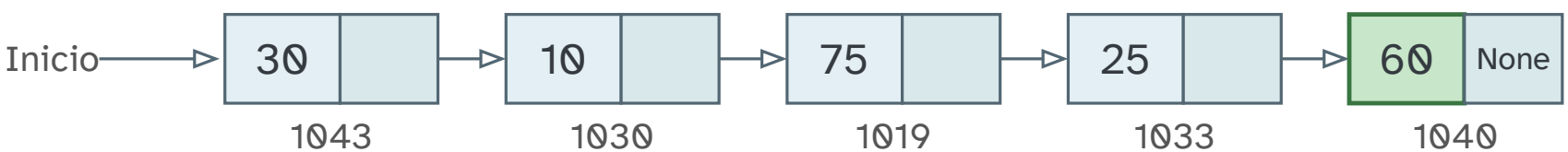
El nuevo nodo, con valor 50, ocupa la dirección 1024 y apunta a 1019. El nodo con valor 10 ahora apunta a 1024. Los demás no se mueven.

Inserción



Si solo conocemos el primer nodo, insertar al final requiere recorrer toda la lista.

Inserción



El último nodo pasa a apuntar al nuevo nodo, que contiene el 60. No desplazamos los valores anteriores.

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)
```

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)

        if indice == 0:
            nodo_nuevo.siguiente = self.primer_nodo
            self.primer_nodo = nodo_nuevo
            return None
```

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)

        if indice == 0:
            nodo_nuevo.siguiente = self.primer_nodo
            self.primer_nodo = nodo_nuevo
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0
```

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)

        if indice == 0:
            nodo_nuevo.siguiente = self.primer_nodo
            self.primer_nodo = nodo_nuevo
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1
```

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)

        if indice == 0:
            nodo_nuevo.siguiente = self.primer_nodo
            self.primer_nodo = nodo_nuevo
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None:
            raise IndexError("Índice fuera de rango")
```

Inserción

```
    def insertar(self, indice, valor):
        if indice < 0:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo = Nodo(valor)

        if indice == 0:
            nodo_nuevo.siguiente = self.primer_nodo
            self.primer_nodo = nodo_nuevo
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None:
            raise IndexError("Índice fuera de rango")
        nodo_nuevo.siguiente = nodo_actual.siguiente
        nodo_actual.siguiente = nodo_nuevo
```

Eliminación

También consiste en actualizar enlaces.

Eliminación

También consiste en actualizar enlaces.

Para eliminar al inicio:

```
self.primer_nodo = self.primer_nodo.siguiente
```

Eliminación

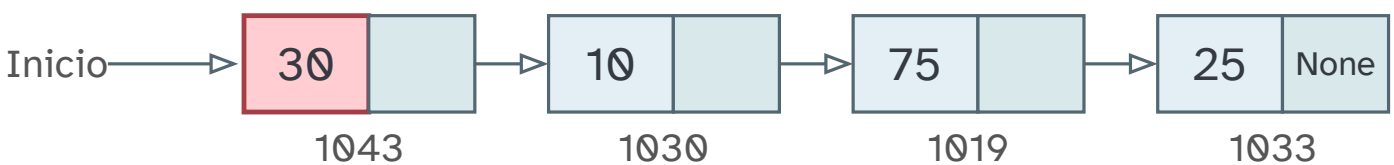
También consiste en actualizar enlaces.

Para eliminar al inicio:

```
self.primer_nodo = self.primer_nodo.siguiente
```

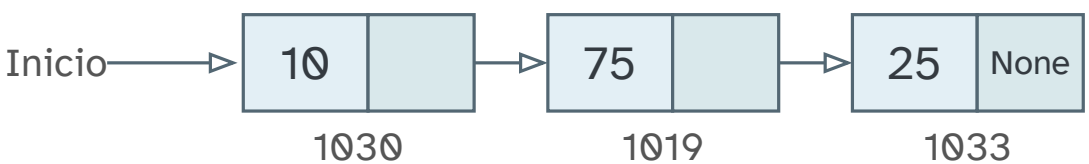
El segundo nodo pasa a ser el primero. El costo es $O(1)$.

Eliminación



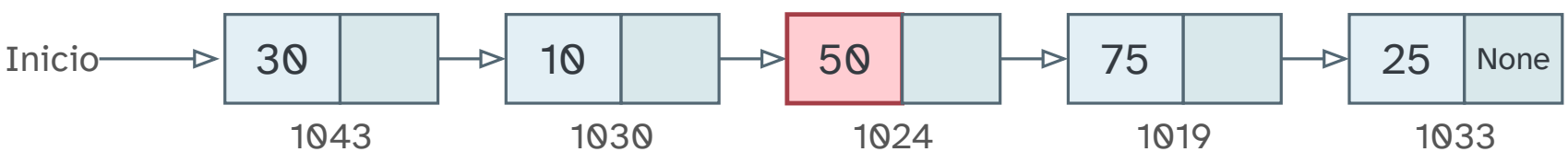
Para eliminar al inicio, hacemos que el segundo nodo pase a ser el primero.

Eliminación



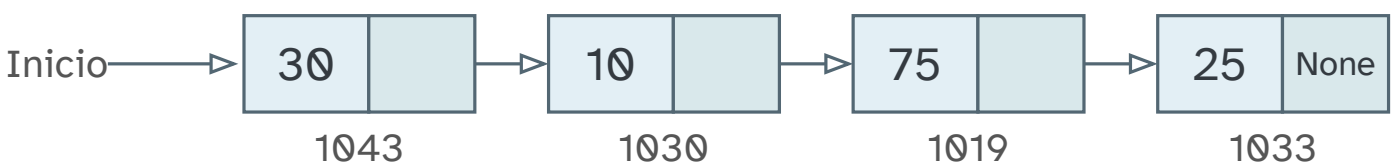
Solo actualizamos el inicio de la lista. El costo es de orden O(1).

Eliminación



Ahora queremos eliminar el 50 de una posición intermedia.

Eliminación



El nodo anterior pasa a apuntar al siguiente del que queremos eliminar. No desplazamos los demás valores.

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")
```

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")

        if indice == 0:
            self.primer_nodo = self.primer_nodo.siguiente
            return None
```

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")

        if indice == 0:
            self.primer_nodo = self.primer_nodo.siguiente
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0
```

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")

        if indice == 0:
            self.primer_nodo = self.primer_nodo.siguiente
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1
```

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")

        if indice == 0:
            self.primer_nodo = self.primer_nodo.siguiente
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None or nodo_actual.siguiente is None:
            raise IndexError("Índice fuera de rango")
```

Eliminación

```
    def eliminar(self, indice):
        if indice < 0 or self.primer_nodo is None:
            raise IndexError("Índice fuera de rango")

        if indice == 0:
            self.primer_nodo = self.primer_nodo.siguiente
            return None

        nodo_actual = self.primer_nodo
        indice_actual = 0

        while nodo_actual is not None and indice_actual < (indice - 1):
            nodo_actual = nodo_actual.siguiente
            indice_actual += 1

        if nodo_actual is None or nodo_actual.siguiente is None:
            raise IndexError("Índice fuera de rango")
        nodo_siguiente_al_eliminado = nodo_actual.siguiente.siguiente
        nodo_actual.siguiente = nodo_siguiente_al_eliminado
```

Enlazando las partes

Nuestra clase `ListaEnlazada` reúne los métodos que acabamos de definir.

```
lista = ListaEnlazada(nodo1)  # 30 → 10 → 75 → 25
lista.leer(3)                 # 25
lista.buscar(75)              # 2
```

Enlazando las partes

Nuestra clase `ListaEnlazada` reúne los métodos que acabamos de definir.

```
lista = ListaEnlazada(nodo1)  # 30 → 10 → 75 → 25
lista.leer(3)                 # 25
lista.buscar(75)              # 2
```

```
lista.insertar(2, 50)         # 30 → 10 → 50 → 75 → 25
lista.leer(2)                 # 50
lista.eliminar(2)             # 30 → 10 → 75 → 25
lista.leer(2)                 # 75
```

Análisis de la eficiencia

Operación	Arreglo	Lista enlazada
Lectura	$O(1)$	$O(N)$
Búsqueda	$O(N)$	$O(N)$
Inserción	$O(N)$	$O(N)$
Eliminación	$O(N)$	$O(N)$

Análisis de la eficiencia

Operación	Arreglo	Lista enlazada
Lectura	$O(1)$	$O(N)$
Búsqueda	$O(N)$	$O(N)$
Inserción	$O(N)$	$O(N)$
Eliminación	$O(N)$	$O(N)$

- Arreglo: insertar y eliminar al final es $O(1)$.*
- Lista enlazada: insertar y eliminar al inicio es $O(1)$.

*En un arreglo dinámico, la inserción al final es $O(1)$ amortizado.

Análisis de la eficiencia

Entonces... ¿por qué usaríamos una lista enlazada?

Análisis de la eficiencia

Entonces... ¿por qué usaríamos una lista enlazada?

> Si ya conocemos el nodo anterior, insertar o eliminar solo requiere actualizar enlaces: $O(1)$.

Análisis de la eficiencia

Entonces... ¿por qué usaríamos una lista enlazada?

> Si ya conocemos el nodo anterior, insertar o eliminar solo requiere actualizar enlaces: $O(1)$.

El costo de recorrer la lista es distinto del costo de modificar los enlaces.

Análisis de la eficiencia

Por ejemplo, queremos eliminar elementos erróneos de una secuencia.

- Tenemos que recorrerla tanto con un arreglo como con una lista enlazada.

Análisis de la eficiencia

Por ejemplo, queremos eliminar elementos erróneos de una secuencia.

- Tenemos que recorrerla tanto con un arreglo como con una lista enlazada.
- En el arreglo, cada eliminación puede desplazar muchos elementos.

Análisis de la eficiencia

Por ejemplo, queremos eliminar elementos erróneos de una secuencia.

- Tenemos que recorrerla tanto con un arreglo como con una lista enlazada.
- En el arreglo, cada eliminación puede desplazar muchos elementos.
- En la lista enlazada, conservando el nodo anterior, actualizamos los enlaces sin desplazar valores.

Listas doblemente enlazadas

Listas doblemente enlazadas

En una lista enlazada simple, cada nodo conoce al siguiente.

Listas doblemente enlazadas

En una lista enlazada simple, cada nodo conoce al siguiente.

En una lista doblemente enlazada, cada nodo tiene dos referencias:

- Una al nodo anterior.
- Otra al nodo siguiente.

Listas doblemente enlazadas

En una lista enlazada simple, cada nodo conoce al siguiente.

En una lista doblemente enlazada, cada nodo tiene dos referencias:

- Una al nodo anterior.
- Otra al nodo siguiente.

La lista mantiene referencias tanto al primer nodo como al último.

Listas doblemente enlazadas

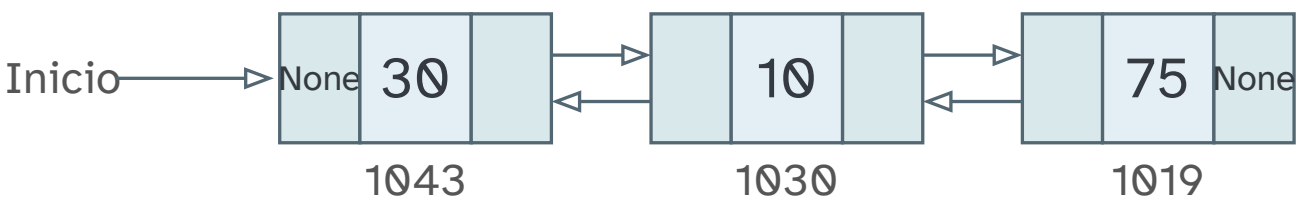
```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.anterior = None
        self.siguiente = None
```

Listas doblemente enlazadas

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.anterior = None
        self.siguiente = None

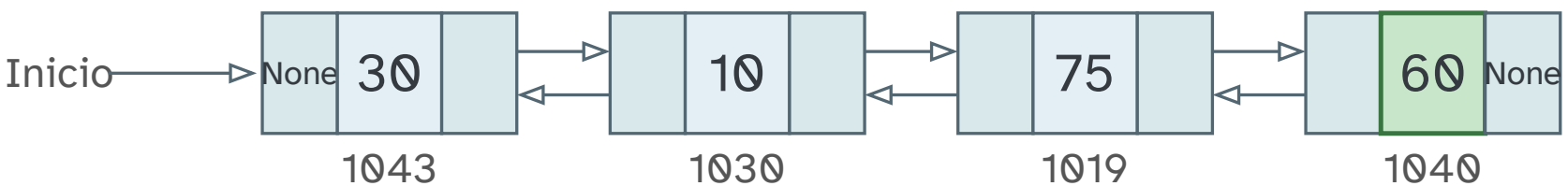

class ListaDoblementeEnlazada:
    def __init__(self, primer_nodo=None, ultimo_nodo=None):
        self.primer_nodo = primer_nodo
        self.ultimo_nodo = ultimo_nodo
```

Listas doblemente enlazadas



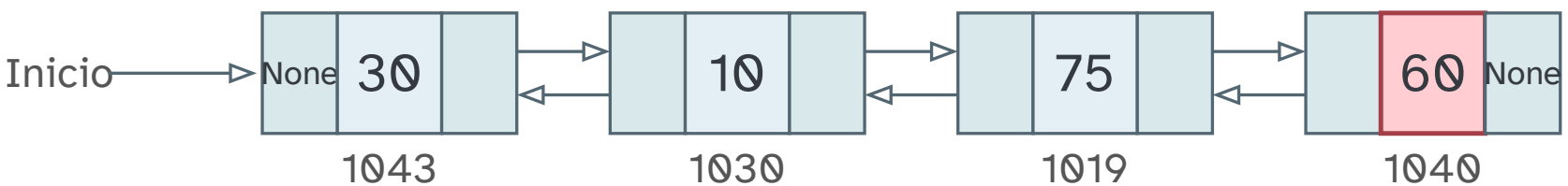
Cada nodo tiene dos referencias: una al anterior y otra al siguiente. El primero no tiene anterior. El último no tiene siguiente.

Listas doblemente enlazadas



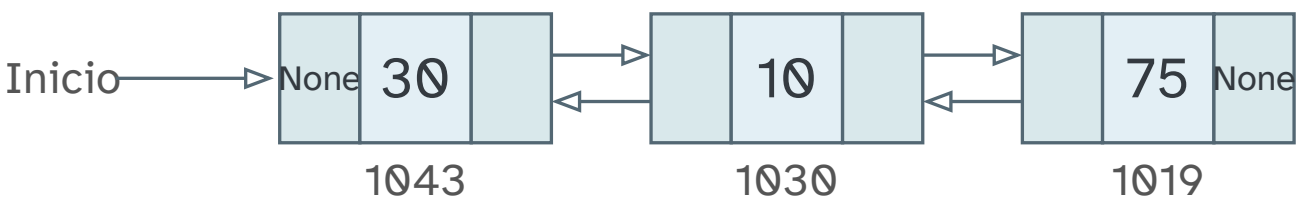
Si mantenemos referencias al primero y al último, podemos insertar al final en tiempo constante. Actualizamos los enlaces en ambas direcciones.

Listas doblemente enlazadas



También podemos eliminar el último nodo sin recorrer la lista desde el inicio.

Listas doblemente enlazadas



Su anterior pasa a ser el último. Su referencia siguiente queda vacía.

Listas doblemente enlazadas

Conocemos ambos extremos de la secuencia.

- Accedemos al primer o al último nodo en $O(1)$.

Listas doblemente enlazadas

Conocemos ambos extremos de la secuencia.

- Accedemos al primer o al último nodo en $O(1)$.
- Insertamos o eliminamos al inicio en $O(1)$.

Listas doblemente enlazadas

Conocemos ambos extremos de la secuencia.

- Accedemos al primer o al último nodo en $O(1)$.
- Insertamos o eliminamos al inicio en $O(1)$.
- También insertamos o eliminamos al final en $O(1)$.

Listas doblemente enlazadas

Conocemos ambos extremos de la secuencia.

- Accedemos al primer o al último nodo en $O(1)$.
- Insertamos o eliminamos al inicio en $O(1)$.
- También insertamos o eliminamos al final en $O(1)$.

> Los enlaces al nodo anterior permiten retroceder desde el último.

Pilas como una lista enlazada

El primer nodo representa la cima de la pila.

Pilas como una lista enlazada

El primer nodo representa la cima de la pila.

- Insertar: agregamos un nuevo primer nodo.
- Extraer: hacemos que el segundo nodo pase a ser el primero.
- Ambas operaciones son $O(1)$.

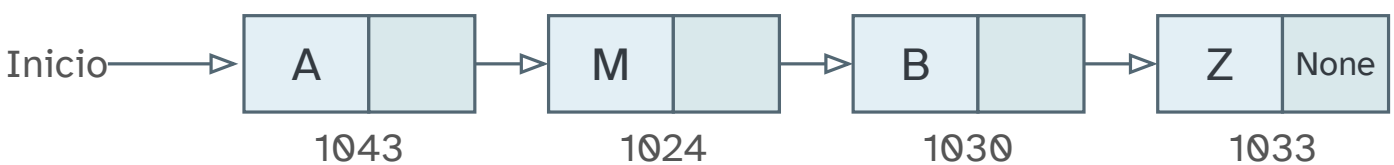
Pilas como una lista enlazada

El primer nodo representa la cima de la pila.

- Insertar: agregamos un nuevo primer nodo.
- Extraer: hacemos que el segundo nodo pase a ser el primero.
- Ambas operaciones son $O(1)$.

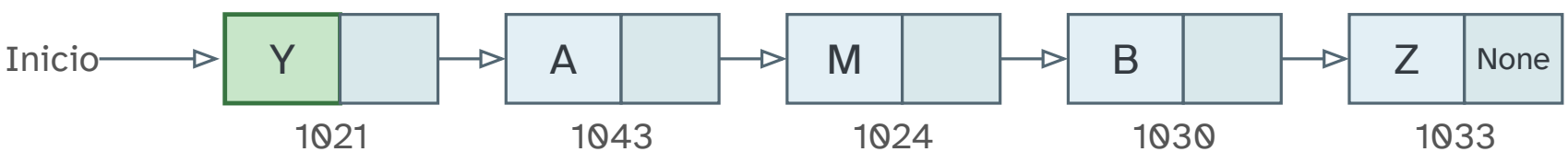
> La misma pila, con el mismo comportamiento LIFO, puede tener distintas implementaciones.

Pilas como una lista enlazada



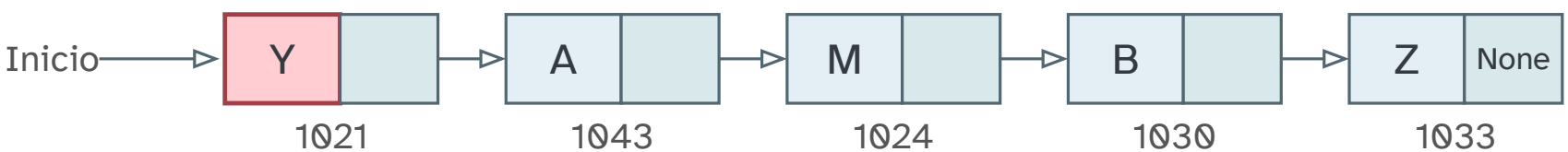
El primer nodo representa la cima de la pila.

Pilas como una lista enlazada



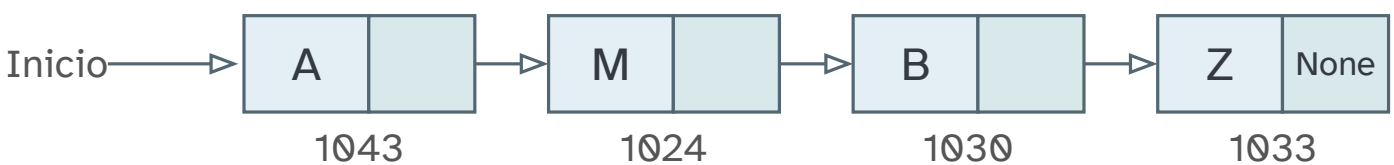
Insertar: agregamos un nuevo primer nodo.

Pilas como una lista enlazada



Extraer: quitamos el primer nodo. El último en entrar es el primero en salir.

Pilas como una lista enlazada



El segundo nodo pasa a ser el primero. Ambas operaciones son de orden O(1).

Pilas como una lista enlazada

Con un arreglo dinámico, agregar al final puede requerir ampliar su capacidad y copiar los elementos.

Pilas como una lista enlazada

Con un arreglo dinámico, agregar al final puede requerir ampliar su capacidad y copiar los elementos.

Con una lista enlazada no necesitamos un nuevo bloque contiguo para toda la pila.

Pilas como una lista enlazada

Con un arreglo dinámico, agregar al final puede requerir ampliar su capacidad y copiar los elementos.

Con una lista enlazada no necesitamos un nuevo bloque contiguo para toda la pila.

> ***Trade-offs...***
>
> Evitamos copiar toda la secuencia al crecer, pero cada nodo también necesita memoria para su enlace.

Colas con una lista doblemente enlazada

Podemos insertar al final y extraer desde el inicio en $O(1)$.

Colas con una lista doblemente enlazada

Podemos insertar al final y extraer desde el inicio en $O(1)$.

En la cola basada en un arreglo:

- Insertar al final es $O(1)$ amortizado.
- Extraer del inicio es $O(N)$: desplazamos los elementos restantes.

Colas con una lista doblemente enlazada

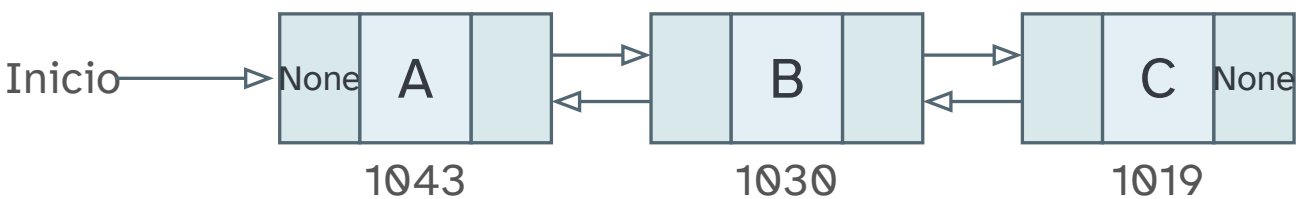
Podemos insertar al final y extraer desde el inicio en $O(1)$.

En la cola basada en un arreglo:

- Insertar al final es $O(1)$ amortizado.
- Extraer del inicio es $O(N)$: desplazamos los elementos restantes.

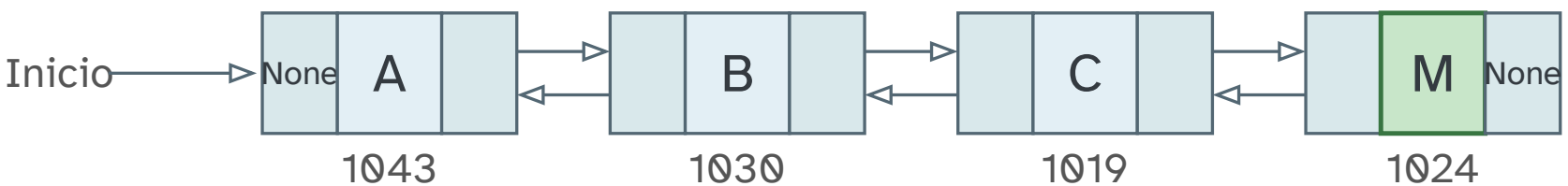
La implementación concreta afecta el costo de las operaciones.

Colas con una lista doblemente enlazada



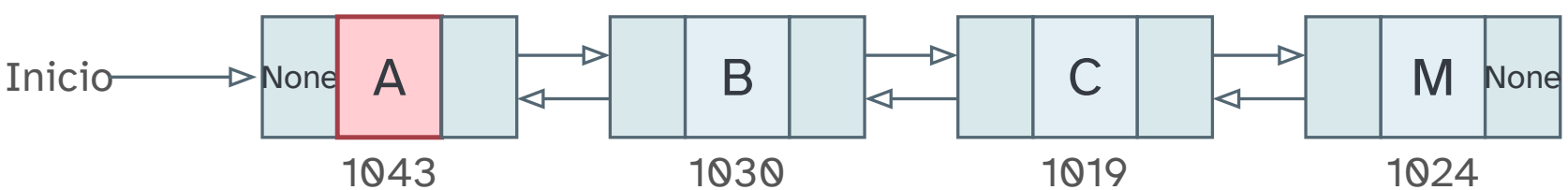
Mantenemos referencias al primer y al último nodo: el frente y el final de la cola.

Colas con una lista doblemente enlazada



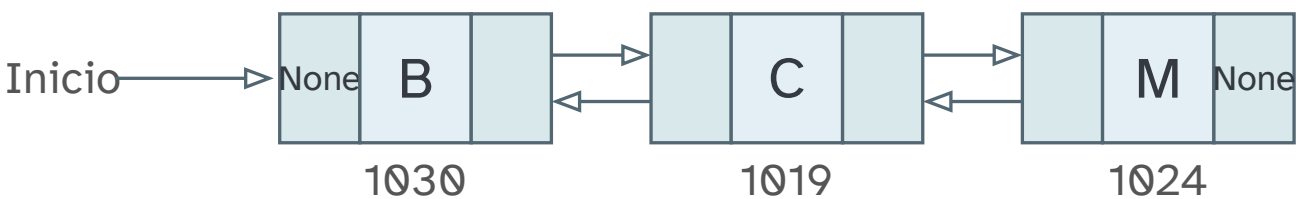
Insertamos M al final usando la referencia al último nodo.

Colas con una lista doblemente enlazada



Extraemos A desde el frente: fue el primero en llegar.

Colas con una lista doblemente enlazada



B pasa a ser el primero. Insertar al final y extraer desde el inicio son operaciones de orden O(1).

Colas con una lista doblemente enlazada

¿Necesitamos siempre enlaces en ambas direcciones?

Colas con una lista doblemente enlazada

¿Necesitamos siempre enlaces en ambas direcciones?

Una lista simplemente enlazada también sirve si mantiene una referencia al último nodo.

Colas con una lista doblemente enlazada

¿Necesitamos siempre enlaces en ambas direcciones?

Una lista simplemente enlazada también sirve si mantiene una referencia al último nodo.

- Agregamos al final usando esa referencia.
- Extraemos desde el primer nodo.
- Ambas operaciones son $O(1)$.

Gracias